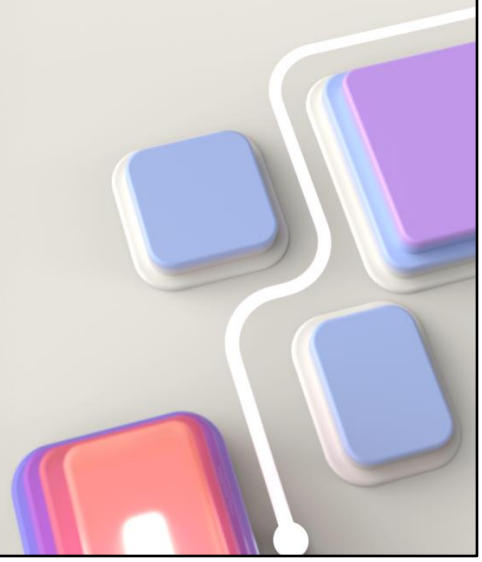
Microsoft

PL-400T00 Microsoft Power Platform Developer

Advanced techniques in canvas apps



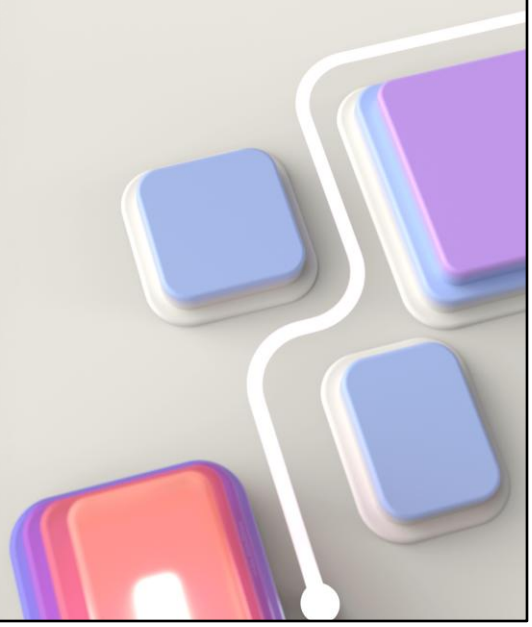
Modules

- Use imperative development techniques for canvas apps
- Perform custom updates in a canvas app
- Use Dataverse choice columns with formulas
- Work with relational data in a canvas app
- Work with data source limits (delegation limits) in canvas apps
- Performance in canvas apps

Modules

1. Use imperative development techniques for canvas apps
2. Perform custom updates in a canvas app
3. Use Dataverse choice columns with formulas
4. Work with relational data in a canvas app
5. Work with data source limits (delegation limits) in a canvas app
6. Performance in canvas apps

Module 1: Use imperative development techniques for canvas apps



Learning Objectives

After completing this module, you will be able to:

1. Understand imperative vs. declarative development
2. Understand the variables in Power Apps
3. Understand when to utilize each of the three different types of variables

Microsoft Learn module

Use imperative development techniques for canvas apps in Power Apps

https://learn.microsoft.com/training/modules/use-imperative-dev-techniques-powerapps-canvas-app

Link to student courseware on Learn

Use imperative development techniques for canvas apps in Power Apps

https://learn.microsoft.com/training/modules/use-imperative-dev-techniques-powerapps-canvas-app

The three types of variables in Power Apps

In your app, you can use variables.

- **Global variables** – The most traditional type of variable. You use the **Set** function to create and set its value. Then you can reference its values anywhere within your app. A common use is to store a user's DisplayName when the app loads and then reference the variable throughout the app.
- **Context variables** – A context variable is only available on the screen where you create it using the **UpdateContext** function. Context variables are commonly used for functionality that controls a pop-up screen, for example, where you want to use the same variable name on multiple screens but maintain the value separately.
- **Collections** – A collection is a special type of variable for storing a table of data. You can create the collection manually or by loading another data sources table into it. Collections are available throughout your app, like global variables, and they are created using the **Collect** or **ClearCollect** function.

In your app, you can use variables. Variables allow you to store information that you need to reference in your app temporarily. Examples where you can use them include to keep a running count or list of information, to dynamically manipulate controls, to optimize performance, or other scenarios where you need to store information temporarily. Variables are a key driver for imperative logic in Power Apps because they allow you to "build the sandwich" piece by piece.

To support you in these needs, Power Apps has three different types of variables.

- Global variables -- The most traditional type of variable. You use the Set function to create and set its value. Then you can reference its values anywhere within your app. A common use is to store a user's DisplayName when the app loads and then reference the variable throughout the app.

- Context variables -- A context variable is only available on the screen where you create it using the UpdateContext function. Context variables are commonly used for functionality that controls a pop-up screen, for example, where you want to use the same variable name on multiple screens but maintain the value separately.

- Collections -- A collection is a special type of variable for storing a table of data. You can create the collection manually or by loading another data sources table into it. Collections are available throughout your app, like global variables, and they are created using the Collect or ClearCollect function.

When choosing which type of variable to use, consider where you will use it and the structure of the data you want to store. When in doubt, use a global variable as it has the most flexibility.

How all of the variable types are the same

With Power Apps, variables are easy to use. You do not have to initialize, declare, or type a variable. You create the variable with the appropriate function, and Power Apps does the rest. When you assign the value to a variable Power Apps will automatically determine the type.

It is also important to note if you are new to variables that variables are temporary and only available to the current user in their current session. When the user closes Power Apps, all of the information stored in variables is no longer available. If you need to store information for use later or by other users, then you will need to write that information to a data source. Variables are temporary by nature.

Global variables

A common design pattern in apps is to personalize the app

“Welcome” & User().FullName

Set(varUserDisplayName, User().FullName)

“Welcome” & varUserDisplayName

Global variables are the most commonly used variables because of their flexibility. After you set the variable, you can reference it or update it throughout your app. This allows you to avoid repetitive query for the same information repetitively, to build out the information you need in an imperative way, or sometimes just as a place holder.

Storing information for your user

A common design pattern in apps is to personalize the app. This can be as simple as saying Welcome and displaying the user's name on each screen of the app. With Power Apps, you can show a welcome message and get the user's name in a declarative manner by using the following formula in a Label control.

"Welcome " & User().FullName

This formula displays the string Welcome and then queries Azure Active Directory for the user's DisplayName property and displays it as text. But if you include that function on every screen then each time a screen opens Power Apps has to query that data from Azure AD directly. This creates repetitive calls to the network that slow down your app.

A better approach would be to store that information in a global variable when the app opens, and then reference that variable throughout your app. You could do this by Modifying the **OnStart** property of the app with the following formula.

Set(varUserDisplayName, User().FullName) Now for your Label control, you would change the formula to the following.

"Welcome " & varUserDisplayName This formula gets you the same output as the previous formula, but instead of having to go back to Azure AD on every screen, Power Apps can reference the value stored in the variable.

Tracking status in a variable

In a declarative mindset, you might hide or show controls based on a query for data

```
CountRows(Filter(InvoiceTable, CustomerNumber = ThisCustomersNumber
And Status = "Outstanding")) > 3
```

```
Set(varOutstandingExceeded, CountRows(Filter(InvoiceTable, CustomerNumber =
ThisCustomersNumber And Status = "Outstanding")) > 3)
```


In a declarative mindset, you might hide or show controls based on a query for data. For example, if you had an app for managing customer's orders, you might have a warning icon that displays only if the customer has more than three outstanding invoices. In addition to the warning, you might have a requirement to get approval from a manager if the customer would like to submit a new order when they have more than three outstanding invoices. This approval workflow will start by the user selecting an approval button.

With a declarative mindset, you would set the Visible property for the warning icon to the following.

If that is true, then the icon displays, and if it is false the icon does not display. You would then repeat that same formula on the Visible property of the Approval button.

The problem is this becomes a complex formula that you maintain in two different locations, and that query will generate duplicate network traffic, processing in the app, and processing on the data source.

A better approach is only to run the complex call once, store the result in a variable, and then use that variable to control the Visible property of each control.

To do this, configure the **OnVisible** property of the screen to set the variable.

The variable **varOutstandingExceeded** is either true or false based on the result of the formula. Now set the **Visible** property of the icon and button control to **varOutstandingExceeded**.

No additional formula or functions are necessary. This is because those controls accept either true or false for the **Visible** property and the variable will be either true or false. Based on your **Set** function in the **OnVisible** property of the screen, Power Apps will set the type of variable to Boolean and set the value to true or false based on the result of the formula.

Small changes like this make your app both more performant and easier to maintain. You should incorporate variables anytime you are repetitively retrieving information that is not going to change while you are using it.

Contextual variables

Contextual variables are similar to global variables except they are only referenced on the screen where you create them

```
Set(varCount, 1);Set(varActive, true);Set(varName, User().FullName)
```

```
UpdateContext({varCount: 1, varActive: true, varName: User().FullName})
```

```
UpdateContext({varShowPopUp: true})
```


Contextual variables are similar to global variables except they are only referenced on the screen where you create them. Although it's not possible to set the user's name to a variable to reference throughout your app, there are still advantages to the fact that contextual variables cannot be used on other screens.

Sometimes you have functionality you want to use on multiple screens that is variable driven. For example, many apps use pop-up dialog boxes to confirm things like deleting a row. A common way to implement this is to set a Contextual variable to true when the user selects the delete button. You do that by setting the **OnSelect** property of the button to the following.

You then set the **Visible** property of the pop-up controls to **varShowPopUp**. This is similar to the example from the global variables. The major difference is reusability. If you copy the controls (using Ctrl+C) to an additional screen, then you will have two instances of **varShowPopUp**. These two instances use the same name, but can have different values. The value of **varShowPopUp** on screen1 does not affect the value of **varShowPopUp** on screen2 because each contextual variable, even when they have the same name, are scoped to the screen they are on.

Typically reusing variable names like this is not recommended because it can be confusing, but it's great if you want to reuse functionality independently on different screens.

If you are in doubt about whether you should use global or contextual variables, typically global variables are the default answer. This is because they are available everywhere, making them the most flexible.

One unique behavior of the **UpdateContext** function is that you can declare more than one variable at a time. This is not possible with the **Set** function. To create more than one context variable with a single formula, use a comma between the variables.

To do the same thing with Global variables, you would use the following.

This is not a reason to sway your choice of variable types but a useful concept to remember. In the next unit, you will learn about storing tables of data in a collection variable.

Using collections to increase performance

The most common reason for using collections is to optimize performance by reducing calls to the same table in a data source.

- The Collect function is not delegable. This means by default only the first 500 rows from the data source will be retrieved and stored in the collection. For more information about working with delegation, see Work with data source limits (delegation limits) in a Power Apps canvas app.
- Collections are not linked to the data source after you create them. This means changes to the data in the collection do not automatically save to the data source. If you want to update the data source based on your changes to the collection, you will need to build formulas to do so.
- Collections are temporary. When you close the app, the collection and all of its contents are removed. If you need to store collection data, you need to write it to a data source before closing the app.

Collect(collectProjects, Projects)

In the previous two units, you learned how global and contextual variables store single values. The third variable type, collections, allow you to store a table of data. This is ideal when you need to store large amounts of structured data for reuse in your app. This data can come directly from a data source, can be created within the app, or a combination of both.

The most common reason for using collections is to optimize performance by reducing calls to the same table in a data source. For example, if you have a table that stores all of your active projects and you want to reference that list multiple times in your app, then you should consider querying for that data once and storing it in a collection. To store a copy of the **Projects** table from your data source into a collection called **collectProjects**, use the following formula.

This will create a collection named **collectProjects** that will have the same rows and columns as the **Projects** table from your data source. Here are a couple of considerations that you need to understand about using collections:

- The Collect function is not delegable. This means by default only the first 500 rows from the data source will be retrieved and stored in the collection. For more information about working with delegation, see Work with data source limits (delegation limits) in a Power Apps canvas app
- Collections are not linked to the data source after you create them. This means changes to the data in the collection do not automatically save to the data source. If you want to update the data source based on your changes to the collection, you will need to build formulas to do so.
- Collections are temporary. When you close the app, the collection and all of its contents are removed. If you need to store collection data, you need to write it to a data source before closing the app.

A variable can store a single row

This concept applies to global and context variables

Set(varUser, User())

varUser.Email

This concept applies to global and context variables. Collections differ slightly because they are a table made up of one or more rows, meaning storing and retrieving a row is different for a collection.

In the previous units, you learned how to store a single value in a global or context variable. You can also store a row in the variable. When you do this, you can then reference the different columns or columns by using the dot (.) notation.

In this example, you will store the entire user row in a global variable named varUser. To do so, use the following function.

This will store the entire user row in the variable. The user row has 3 columns Email, FullName, and Image. You can retrieve the values of the individual columns using the dot (.) notation. To display the user's email address, add a Label control to the screen and set the text property to:

This example stores the row from an action-based data source. You could also use the LookUp function as a way to retrieve and store a row from a tabular data source, like Microsoft Dataverse, in a variable.

Variables don't auto update

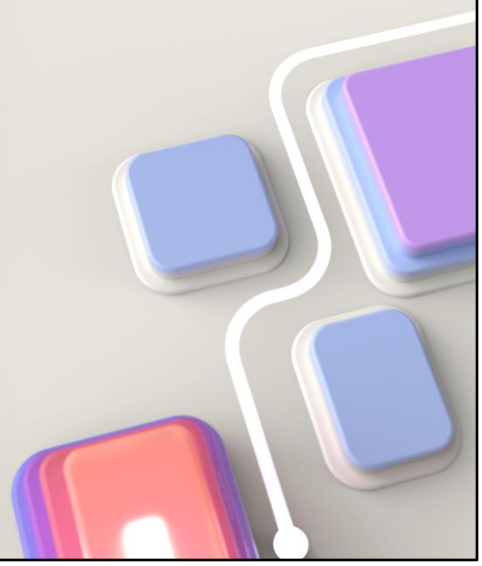
A common point of confusion for people who are new to variables is that variables don't automatically update. For example, they can use a variable to store the number of customer invoices using OnStart for the app. Then in the app, the user creates a new invoice. The variable doesn't distinguish the number of invoices in the system that have changed. The variable will only update when:

•The user closes the app and then opens it again. This causes OnStart to perform the operation to calculate the number of invoices.

•You implement functionality to update the variable after the user creates an invoice.

Be aware of this common point of confusion if you are new to using variables to track data.

Module 2: Perform custom updates in a canvas app



With some Power Apps canvas apps a form is not the solution. This module will focus on how to perform custom updates when your data is not in a form.

In this module, you will:

- Use the Patch function to update your data
- Understand how the Defaults function is used to create new records with Patch
- Utilize the Remove and RemoveIf functions to delete records
- Determine whether to use Clear and Collect or ClearCollect in their scenario

Learning Objectives

After completing this module, you will be able to:

1. Use the Patch function to update your data
2. Understand how the Defaults function is used to create new records with Patch
3. Determine whether to use Clear and Collect or ClearCollect

Microsoft Learn module

Perform custom updates in a Power Apps canvas app

https://learn.microsoft.com/training/modules/perform-custom-updates-powerapps-canvas-app

Link to student courseware on Learn

Perform custom updates in a Power Apps canvas app
https://learn.microsoft.com/training/modules/perform-custom-updates-powerapps-canvas-app

Directly create and edit rows

In this module, you will learn about using the Patch function to create and update your data sources without the use of forms directly.

```
Patch(LoggingTable,
       Defaults(LoggingTable),
      { WhoClicked: User().FullName,
        WhenClicked: Now() } );
```


Sometimes you need something more than forms

When building canvas apps in Power Apps, you go to Galleries to display rows from your data source and Forms to view, create, and edit an individual row, but sometimes forms are not enough. In those scenarios, Power Apps has functions for updating your tabular data sources directly.

Directly create and edit a row

In this module, you will learn about using the **Patch** function to update your data sources without the use of forms directly.

Patch is most often used when you need to take action on the data without user interaction in a repetitive manner, or your app design doesn't allow for the use of forms. For example, if you want to update a logging data source every time a user clicks a button to navigate to another screen you could use the formula for the OnSelect property of the button.

Copy

Patch(LoggingTable, Defaults(LoggingTable), {WhoClicked: User().FullName, WhenClicked: Now()}); Navigate(NextScreen, ScreenTransition.Cover) This formula would create a new row in the data source named **LoggingTable**. The WhoClicked column sets to the **FullName** property of the user who is signed in, and the WhenClicked column sets to the Date and Time of when they clicked the button.

Using Patch to create a row

The Patch function can be used to create a new row in your data source.

- Include the name of the data source you want to edit. This could be a tabular data source (such as Dataverse or SharePoint) or a collection. For the example, you will use **CustomerOrders** as the name of the data source.
- The **Defaults** function returns a row that contains the default values for the data source. By using Defaults with the data source, this notifies Patch to create a new row.
- Include the columns that you want to populate in the new row. Specify the name of the column to update followed by the value to write to that column. For this example, you will update the Region and Country column with a string value.

```
Patch(CustomerOrders, Defaults(CustomerOrders), {Region: "Americas", Country: "Canada"})
```


Using the Patch function to create and edit rows

•15 minutes

The **Patch** function is used to create and edit rows in a data source when using a Form control doesn't meet your needs. Patch is used most often when you need to take action on the data without user interaction, in a repetitive manner, or your app design doesn't allow for the use of forms.

The Patch function can be used to create a new row in your data source. To create a new row, there are three parts to the formula.

1. Include the name of the data source you want to edit. This could be a tabular data source (such as Dataverse or SharePoint) or a collection. For the example, you will use CustomerOrders as the name of the data source.

1. The Defaults function returns a row that contains the default values for the data source. If a column within the data source doesn't have a default value, that property won't be present. By using Defaults with the data source, this notifies Patch to create a new row.

1. Include the columns that you want to populate in the new row. Here you will specify the name of the column to update followed by the value to write to that column. For this example, you will update the Region and Country column with a string value.

The example formula is as follows:

This formula will create a new row in the CustomerOrders data source and will set the Region to Americas and Country to Canada. Notice that you do not define any primary key information (the ID column) that will be updated by the data source according to its settings.

Using Patch to edit a row

It is also possible to edit a row in the data source.

- Include the name of the data source you want to edit. For the example, you will use **CustomerOrders** as the name of the data source.
- The row that you want to edit in the data source. The most common way to specify this row is to use the **LookUp** function to retrieve the row from the data source. Another option if you are using a Gallery and you want to update the current row is to use the **ThisItem** function for referencing the row. For this example, you will use a LookUp function.
- Include the changes that you want to make. Here you will specify the name of the column to update followed by the value to write to that column. For this example, you will update the Region and Country column with a string value.

```
Patch(CustomerOrders, LookUp(CustomerOrders, ID = 1), {Region: "Asia", Country: "China"})
```


It is also possible to edit a row in the data source. To edit a single row, there are three parts to the formula.

1. Include the name of the data source you want to edit. This could be a tabular data source (such as Dataverse or SharePoint) or a collection. For the example, you will use CustomerOrders as the name of the data source.

1. The row that you want to edit in the data source. The most common way to specify this row is to use the LookUp function to retrieve the row from the data source. Another option if you are using a Gallery and you want to update the current row is to use the ThisItem function for referencing the row. For this example, you will use a LookUp function.

1. Include the changes that you want to make. Here you will specify the name of the column to update followed by the value to write to that column. For this example, you will update the Region and Country column with a string value.

The example formula is as follows:

This formula will update the row with an ID of 1 in the CustomerOrders table by setting the Region column to Asia and the Country column to China. If there are existing values in those columns, it will be overwritten.

Updating columns with Patch

The primary logic of most Patch functions is updating the proper columns with the correct information.

- Make sure you are updating all of the required columns from your data source.
- You can update as many or as few of the optional columns as you would like.
- Make sure your column names are spelled and capitalized correctly. Column names are case-sensitive.
- Make sure you are writing the correct data type. For example, if your column in the data source is a number type, then you cannot write a string value to it, even if that string contains a number.

The primary logic of most Patch functions is updating the proper columns with the correct information. This will be the source of most of your troubleshooting of the Patch function. Use the following points to help you work through Patch.

- Make sure you are updating all of the required columns from your data source.
- You can update as many or as few of the optional columns as you would like.
- Make sure your column names are spelled and capitalized correctly. Column names are case-sensitive.
- Make sure you are writing the correct data type. For example, if your column in the data source is a number type, then you cannot write a string value to it, even if that string contains a number.

There are four sources to pass values in your formula to Patch your data source:

- You can hardcode a value. An example is if you want to patch the status of the row with "Pending", your Patch formula would look like:
- Patch(CustomerOrders, Default(CustomerOrders), {Status: "Pending"})
- This formula creates a new row and sets the Status column to the string value of "Pending".
- You can reference a variable. For example, you can store the string "Under Review" in a variable named varStatus with the following formula.
- Set(varStatus, "Under Review")
- Then your Patch formula would be:
- Patch(CustomerOrders, Default(CustomerOrders), {Status: varStatus})
- This formula creates a new row and sets the Status column to the string value of "Under Review".
- You can reference the value from the property of a control. An example would be setting the value from a drop-down menu named Dropdown1 that contained the regions. Your Patch formula would look like:
- Patch(CustomerOrders, Default(CustomerOrders), {Status:Dropdown1.Selected.Value})
- This formula creates a new row and sets the Status column to the value of the selected item in the drop-down menu.
- You can use the output of a formula. An example would be setting the value of the ModifiedBy column using the FullName from the User() function. Your Patch formula would look like:
- Patch(CustomerOrders, Default(CustomerOrders), {ModifiedBy:User().FullName})
- This formula creates a new row and sets the ModifiedBy column to the current user's FullName from Azure Active Directory.

Demo: Patch

DEMO

Delete a row

There are also functions available for deleting one or more rows from your data source.

- **Remove and RemoveIf** – These functions are used to remove or delete rows from the data source.
- **Clear** – Use the **Clear** function to remove all of the rows from a collection.

```
Remove(CustomerOrders, LookUp(CustomerOrders, ID = 1))
```


There are also functions available for deleting one or more rows from your data source. Those functions are:

- Remove and RemoveIf - These functions are used to remove or delete rows from the data source.
- Clear - Use the Clear function to remove all of the rows from a collection.

For example, if you wanted to give the user the ability to delete a row from a Gallery control, add a Trash icon to the Gallery displaying the data source CustomerOrders and then set the OnSelect property of the icon to the following.

This formula would delete the row for the item that was displaying the Trash icon from the CustomerOrders data source. There would be no confirmation, so you might consider implementing a check or pop-up dialog to confirm that the user truly wants to delete the row.

Delete all of the rows in a collection

The Clear function deletes all the rows of a collection.

- All at once – For example, if you are reloading the items in a collection for a drop-down menu when a screen becomes visible, you would want to use **ClearCollect**. One formula would then remove the old rows and add the new rows.
- Multi-step – For example, if you are using collections to store user inputs like in a shopping cart you can use **Clear** and **Collect**. This is because the user might want to clear their shopping cart without adding a new row.

```
Clear(collectSelectedItems)
```


Delete all of the rows

It is also possible to delete all of the rows in a data source. This is most common with collections where you can use the **Clear** function. If you want to delete all of the rows from a data source, you can use **RemoveIf**.

The Clear function deletes all the rows of a collection. The columns of the collection will remain. The only input you pass to the function is the collection name.

For example, you could use the following formula to delete all of the rows from a collection.

This formula will delete all of the rows from the collectSelectedItems collection without changing the columns of the collection.

You will typically see this type of formula when you want to clear out the collection without having to redefine it, like in the case of a reset button or selecting a new order. Note that when working on collections you also have the ClearCollect function.

The ClearCollect function deletes all the rows from a collection and then adds a different set of rows to the same collection. With a single function, ClearCollect offers the combination of Clear and then Collect.

All three functions have their place. One way to think about whether you want to use Clear and Collect versus ClearCollect is when the clearing of the collection happens compared to when you want to add rows back. Here are two examples to illustrate this:

- All at once - For example, if you are reloading the items in a collection for a drop-down menu when a screen becomes visible, you would want to use ClearCollect. One formula would then remove the old rows and add the new rows.
- Multi-step - For example, if you are using collections to store user inputs like in a shopping cart you can use Clear and Collect. This is because the user might want to clear their shopping cart without adding a new row.

Bulk changes to rows

Patch and Remove are both functions that are used to affect one row

- Use the **ForAll** function, to loop through a table of data and run a Patch or Remove function for each row in the table.
- Use the **Collect** function to write from one table to another. Each row of the source table is added as a separate row to the target table.

Patch and Remove are both functions that are used to affect one row. If you need to affect change on more than one row, there are two options:

- Use the ForAll function, which was covered in the previous module, to loop through a table of data and run a Patch or Remove function for each row in the table.

- Use the Collect function to write from one table to another. Each row of the source table is added as a separate row to the target table.

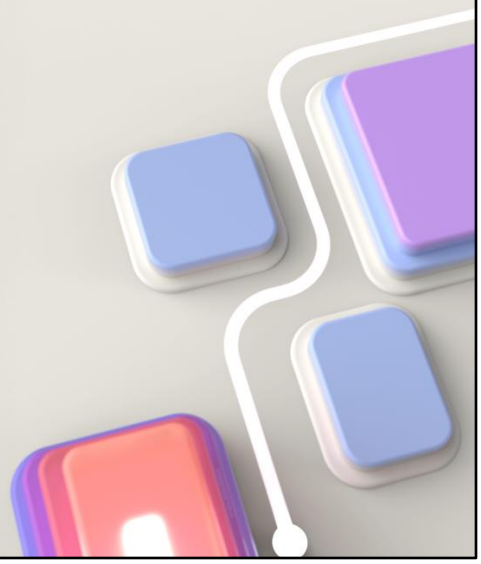
These topics are covered in other Power Apps learning paths and are not covered in this learning path.

Collections are data sources

It's important to remember that these functions can use a collection as their target. Patch, Remove, and RemoveIf can all be used to modify both tabular data sources and collections. As you build more complex apps storing data in collections and working with those items is very common, these functions will be a big part of that manipulation.

The remainder of this module will refer to updating a data source. Remember that a data source can either be a tabular data source or a collection unless stated otherwise.

Module 3: Use Dataverse choice columns with formulas



Learning Objectives

After completing this module, you will be able to:

1. Discover the choice field basics.
2. Learn when to use choice or lookups.
3. Filter data on choice values.

Microsoft Learn module

Use Dataverse choice columns with formulas

https://learn.microsoft.com/training/modules/choice-columns-formulas

Link to student courseware on Learn

Use Dataverse choice columns with formulas
https://learn.microsoft.com/training/modules/choice-columns-formulas

Filter with Choice columns

Items in Choice columns are a key/value pair

- Value is stored on row
- Label is stored in metadata
- Labels are available in canvas app
- For Dataverse column can filter as follows

```
Filter(
  Accounts,
  Category = 'Category (Accounts)'.'Preferred Customer'
)
```


When you have a Dataverse table with choice column, you'll often want to filter the data by using a choice column.

The most common filtering scenarios are:

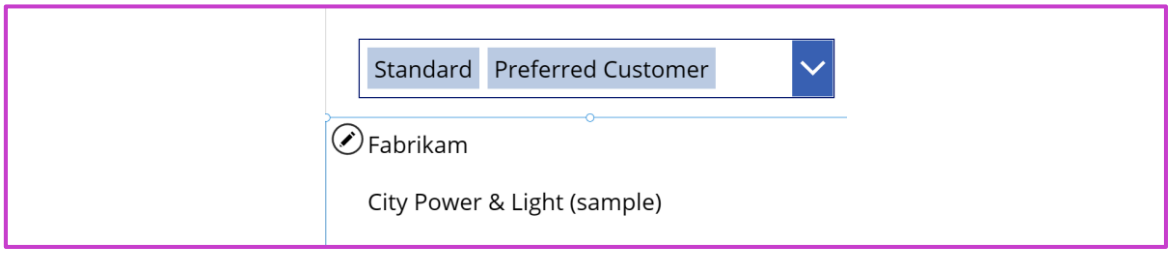
- Filter the table rows for display in a gallery.
- Have a dropdown menu or combo box control with the list of choice values, and then let the user select one or more. Then, you can use the selected values to filter the table rows that you show in the gallery.

For example, if you had a **Category** choice field on the Accounts table, you could use the following logic to filter only the preferred customers:

Filter using a Choice dropdown

Use Choices() function to display list of items from a choice column

Filter(Accounts, Category in ComboBoxCategory.SelectedItems)



Use Choices() function to display list of items from a choice column

The Choices() function prepares a list of values for your user to select from by using the metadata for the choice column **Accounts.Category**.

The **Items** formula for the gallery to include using the combo box **SelectedItems** property.

Filter(Accounts, Category in ComboBoxCategory.SelectedItems)

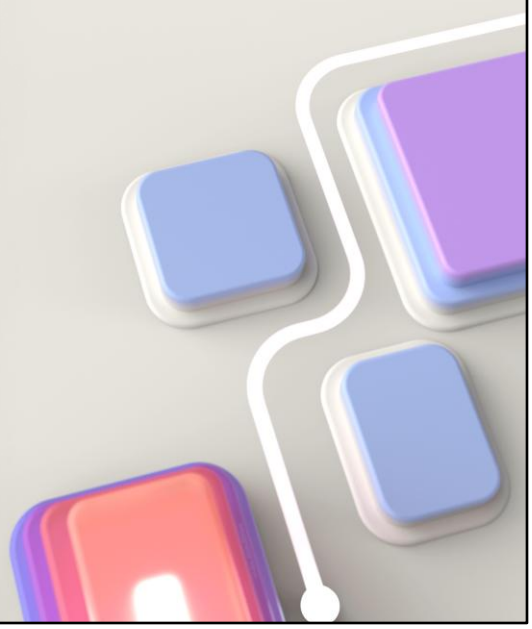
Patch with Choice columns

Choice columns require the choice set name when using Patch

```
Patch(Accounts, ThisItem, {Category: Category.'Preferred Customer'})
```


If your table column uses a Choice set, when you use Patch, you'll need to prefix your value with the Choice set name, else you'll get an 'OptionSetValue' error. Y

Module 4: Work with relational data in a canvas app



Next we will talk about Building a data model

Learning Objectives

After completing this module, you will be able to:

1. Work with relational data in a Power Apps canvas app
2. Reduce complexity in your data model with Dataverse table relationships

Microsoft Learn module

Work with relational data in a Power Apps canvas app

https://learn.microsoft.com/training/modules/work-with-relational-data-powerapps-canvas-app

Reduce complexity in your data model with Dataverse table relationships

https://learn.microsoft.com/training/modules/reduce-complexity-dataverse-table

Link to student courseware on Learn

Work with relational data in a Power Apps canvas app

https://learn.microsoft.com/training/modules/work-with-relational-data-powerapps-canvas-app

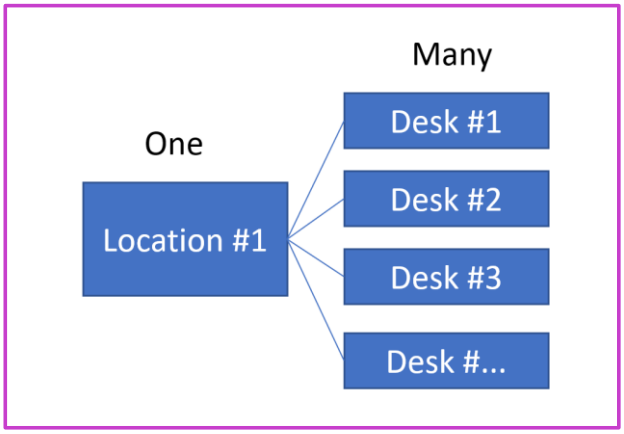
Reduce complexity in your data model with Dataverse table relationships

https://learn.microsoft.com/training/modules/reduce-complexity-dataverse-table

One-to-many relationships

One-to-many relationships are the most common Dataverse relationships that you will work with

The following example is a one-to-many relationship from the Location table to the Desk table



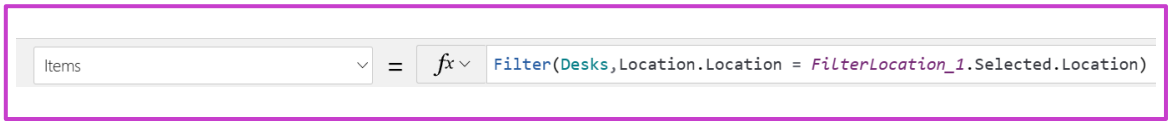
One-to-many relationships are the most common Dataverse relationships that you will work with.

This unit continues the scenario uses a shared workspaces (*hot desking*) solution for Contoso.

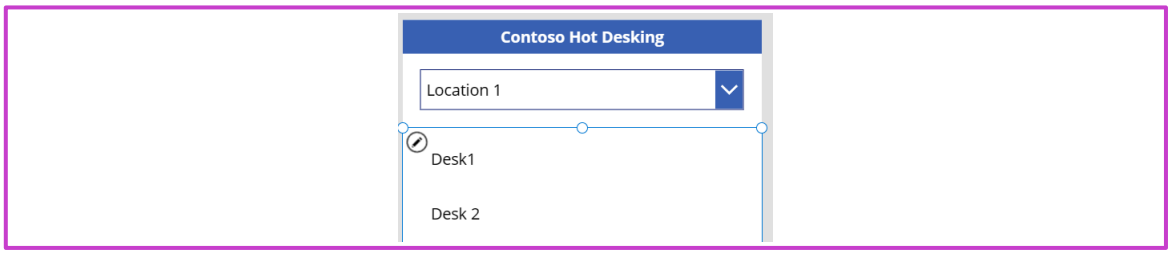
To help explain how to work with relationships in a canvas app, the ensuing examples will use the relationship between the Location and Desk tables. The following diagram is a visualization of the relationship and the corresponding data.

Filter using a One-to-many relationship

Use Filter() function to display list of desks for a selected location



Note joining on the record not the Guid using the dot notation

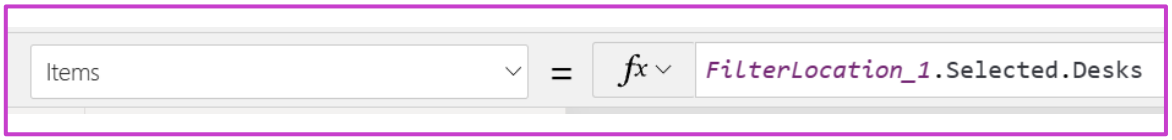


Use Filter() function to display list of desks for a selected location

Note joining on the record not the Guid using the dot notation

Get related rows using a One-to-many relationship

Use the dot notation to retrieve related rows



Use the dot notation to retrieve related rows

Get row from a Many-to-one relationship

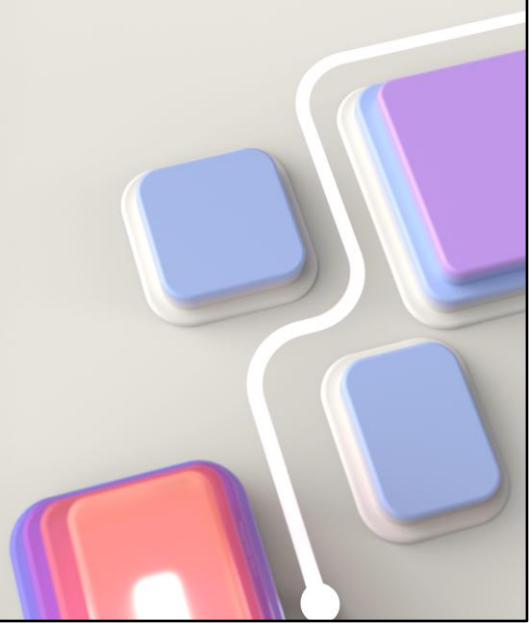
Use the dot notation on the lookup column to retrieve related row

Instead of using the Lookup() formula, you can use the dot notation **on the lookup column** and **reference** ThisItem.Location.Address.

Demo: Relationships in canvas apps

DEMO

Module 5: Work with data source limits (delegation limits) in canvas apps



Learning Objectives

After completing this module, you will be able to:

1. Understand the different limits of different data sources
2. Understand how functions, predicates, and operators all play roles in the limits
3. Use this new understanding to choose the best data source for an app

Microsoft Learn module

Work with data source limits (delegation limits) in a Power Apps canvas app

https://learn.microsoft.com/training/modules/work-with-data-source-limits-powerapps-canvas-app

Link to student courseware on Learn

Work with data source limits (delegation limits) in a Power Apps canvas app

https://learn.microsoft.com/training/modules/work-with-data-source-limits-powerapps-canvas-app

Delegation overview

Before you choose your data source in Power Apps, it's important to understand delegation.

Delegation in action

```
Filter(SharePointList,
ProjectStatus = "Active")
```

When delegation isn't available

Consider delegation when choosing a data source

Before you choose your data source in Power Apps, it's important to understand delegation. By using delegation, Power Apps optimizes its interaction with data sources to minimize the amount of transferred data. Put more simply, Power Apps uses delegation to offload the processing (which includes filtering, searching, and sorting) of data to the data source when available. Delegable processing is dependent on the data source and function being utilized. If you have a lot of data and need to rely on the back-end data source for operations such as filtering, then you might want to consider moving (or replicating) your data into an environment (such as Dataverse) that works well with delegation. To replicate your data from a different source, you can use the Data Integrator to move data into Dataverse.

Delegation in action

There's an example to help you understand delegation. You have a list of 5,000 projects stored in SharePoint. The **ProjectStatus** column stores the values **Active** or **Inactive**. Half (2,500) of the rows are set to each of these values. You could show the list in a gallery and filter it by using this formula.

Filter(SharePointList, ProjectStatus = "Active")

Because the **Filter** function is delegable to a SharePoint data source, Power Apps would send your formula to SharePoint. SharePoint would process all 5,000 rows and return to Power Apps the 2,500 rows for which **ProjectStatus** is set **Active**. Those rows would all be available in your gallery. In this scenario, Power Apps didn't process any data, and only the matching rows were sent from SharePoint to Power Apps, which is efficient.

When delegation isn't available

Some functions cannot be delegated to some data sources. An example of a non-delegable action is the **Search** function against the SharePoint data source. The **Search** function is similar to **Filter**, but you can use **Search** to check across multiple columns and to find partial matches. For example, Search(SharePointList, "Rob", "FirstName","LastName") would return all of the rows where the string "rob" is in either the **FirstName** or the **LastName** column. In this example, Power Apps would return rows for Robert Smith, Rob Jones, and Suzy Robinson.

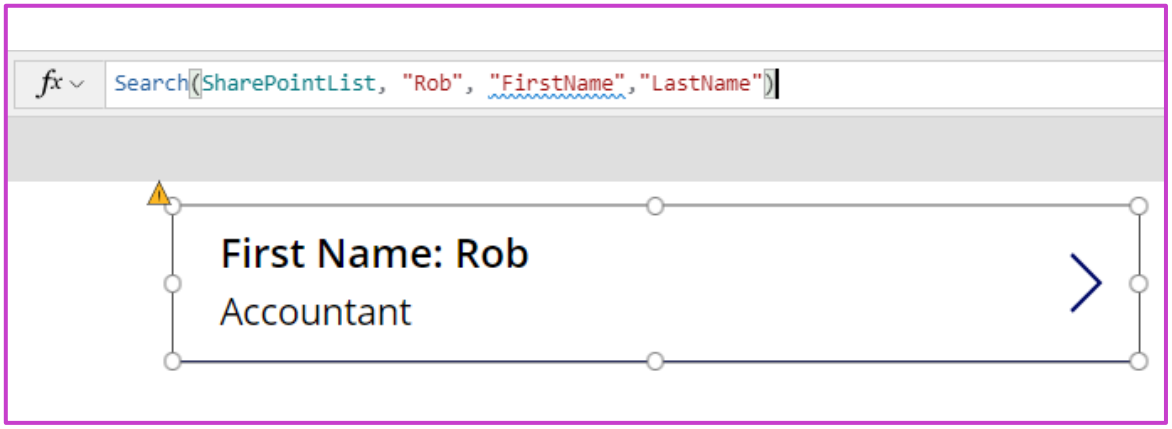
Because the **Search** function isn't delegable to the SharePoint data source, Power Apps has to process the rows locally, which means first the rows are sent from SharePoint to Power Apps. By default, the data source will only return the first 500 rows. It is not the first 500 rows that match your formula, but the first 500 rows in the data source. In this example, when you add this formula to your gallery, SharePoint returns the first 500 rows from the list to Power Apps, and then Power Apps performs the search operation locally. If your list contained 5,000 items, the other 4,500 rows in your data source are not processed or displayed. You can increase the default of 500 rows to a maximum of 2,000 rows in the advanced settings of Power Apps Studio. Under those circumstances, 3,000 rows still would not be processed or displayed.

Consider delegation when choosing a data source

The reason there is an entire module dedicated to delegation is that it is a key consideration when choosing your data source. You need to consider the types of functions you need, like Search, and the amount of data you will have as you chose the best data source for your application.

Delegation warnings

Anytime you use a non-delegable function, Power Apps underlines it with a blue line and displays a yellow warning triangle as shown below.



Delegation warnings, limits, and non-delegable functions

•10 minutes

Power Apps uses visuals to help you, the app maker, understand when delegation is occurring. The maker portal also has one setting you can adjust to increase the amount of data returned when delegation is not possible.

Anytime you use a non-delegable function, Power Apps underlines it with a blue line and displays a yellow warning triangle as shown below.

This gives you a clear visual indicator that delegation is not happening, which means you may not be seeing all of your data. It is important to understand a couple of things about this visual indicator.

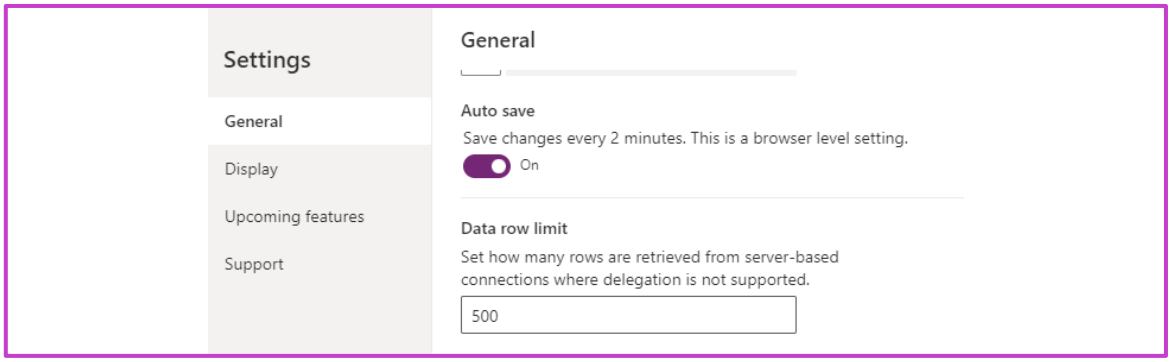
- Power Apps will provide this warning regardless of the size of your data source. Even if your data source only has a few items and delegation isn't technically causing you a problem (remember the first 500 items are returned by default and processed locally) the warning will still show. The warning appears anytime that your formula is not delegated.

- The warning indicator only processes through the first thing that causes delegation. Notice in the above screenshot that only the underlined column "FirstName" is in blue. That is because it was the first item that caused delegation. "LastName" would also cause delegation in this scenario. This can be confusing because people try to troubleshoot what is the difference between FirstName and LastName instead of the real issue, which is the Search function. If you encounter this confusion, rearrange your formula. This will validate and whichever column is first will show the issue.

- This visual indicator is only present when you are in the maker portal, building the app. When a user is running the app, they will not receive any notification that delegation is not occurring and that they may only be seeing partial results. Keep this in mind when designing your app and build accordingly.

Changing the number of rows returned when delegation isn't available

When a formula cannot delegate to the data source, for any reason, then by default the canvas app retrieves the first 500 rows from that data source and then processes the formula locally



When a formula cannot delegate to the data source, for any reason, then by default Power Apps retrieves the first 500 rows from that data source and the processes the formula locally. Power Apps does support adjusting this limit from 1 to 2000. You can adjust this in the Advanced settings.

1. From the Maker portal, select File in the upper-left corner.

1. In the left-most menu, select Settings.

1. Under App settings, select Advanced settings

1. Set the Data row limit for non-delegable queries for any value between 1 to 2000.

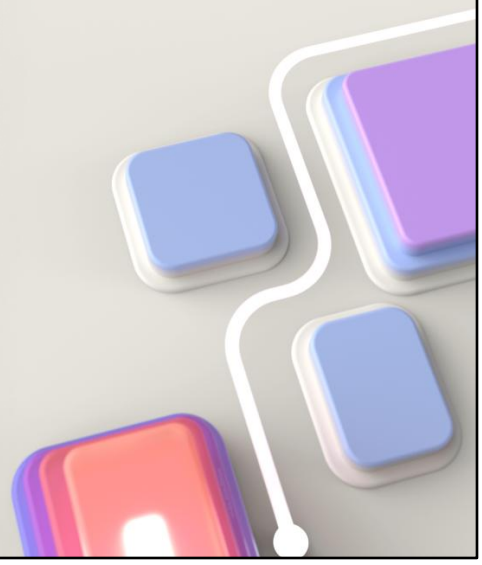
1. After you have set the limit, select the arrow in the upper left to save your change and return to the Maker portal.

There are two primary reasons that you might want to adjust this limit.

- To increase the limit if you are working with data where 500 rows aren't enough, but less than 2000 is. For example, if you have a customer list and you know you will never have more than 1000 customers, then you could design your app to ignore delegation and always return all 1000 rows.

- To decrease the limit to 1 or 10 to help with testing. If you are running into scenarios where you are not sure if a non-delegable function is causing problems with your app, you can lower the limit and then test. If you set the limit to 1 and your gallery is only presenting one row, you know that you had a non-delegable function. This speeds up your troubleshooting process.

Module 6: Performance in canvas apps



This lesson is an add-on for exam coverage purposes

Learning Objectives

After completing this module, you will be able to:

1. Complete testing and performance checks in a Power Apps canvas app
2. Optimize app load time
3. Use Monitor to troubleshoot Power Apps
4. Use Power Apps Instrumentation with Application Insights

Microsoft Learn module

Complete testing and performance checks in a Power Apps canvas app
https://learn.microsoft.com/training/modules/testing-performance-checks-powerapps

Optimize app load time
https://learn.microsoft.com/training/modules/optimize-app-load-time

Use Monitor to troubleshoot Power Apps
https://learn.microsoft.com/training/modules/monitor-to-troubleshoot

Use Power Apps Instrumentation with Application Insights
https://learn.microsoft.com/training/modules/instrumentation-app-insights

Link to student courseware on Learn

Complete testing and performance checks in a Power Apps canvas app

https://learn.microsoft.com/training/modules/testing-performance-checks-powerapps

Optimize app load time

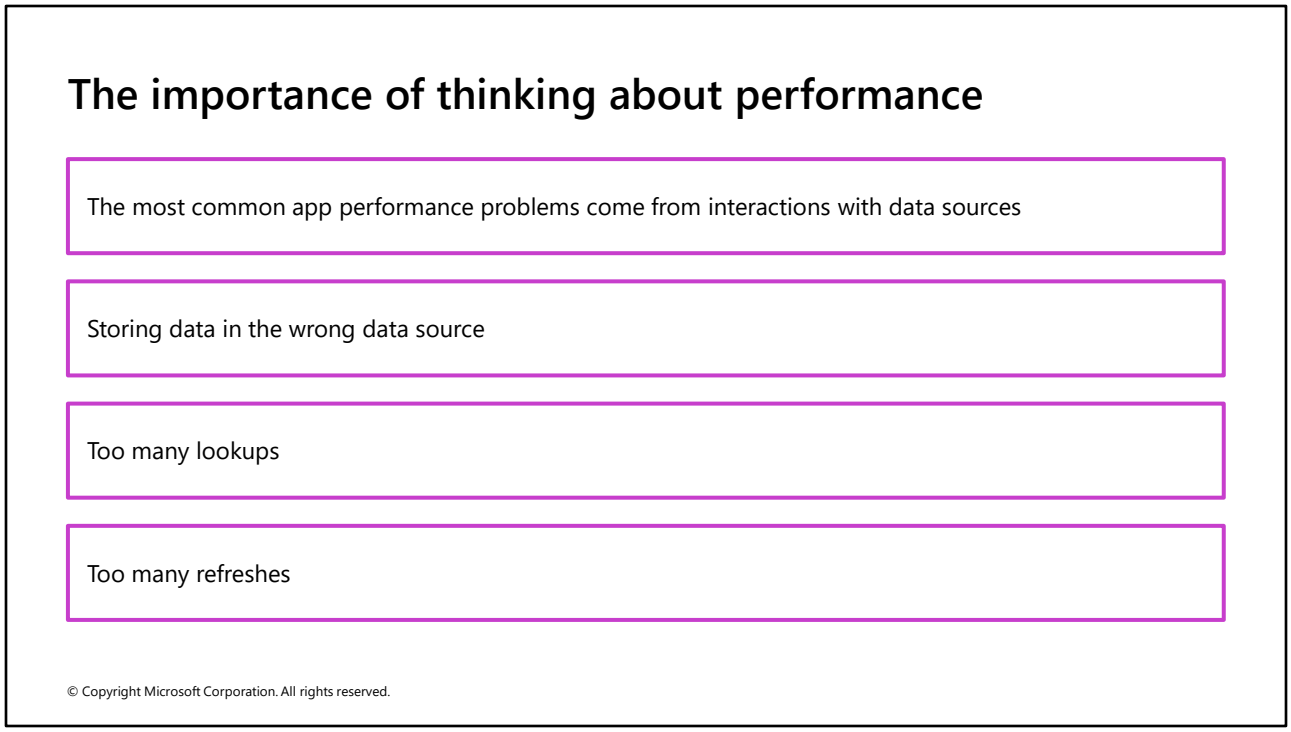
https://learn.microsoft.com/training/modules/optimize-app-load-time

Use Monitor to troubleshoot Power Apps

https://learn.microsoft.com/training/modules/monitor-to-troubleshoot

Use Power Apps Instrumentation with Application Insights

https://learn.microsoft.com/training/modules/instrumentation-app-insights



The importance of thinking about performance

•8 minutes

The performance of an app is important to keep users happy. Apps can go from mediocre to great just based on performance. And sometimes it can be as simple of a change as caching data in a collection or removing redundant calls to the data source.

In this module, you will learn about common performance problems, ways to mitigate their impact, and how to perform testing to discover the issues.

The most common performance bottleneck - Data sources

The most common app performance problems come from interactions with data sources. Almost every app has one or more data source. Power Apps natively supports over 200 different connections to these data sources and using these connections is key to a great app.

Calling these data sources from your app is often the largest bottleneck in your app because of the time it takes to call across the network to the data source, process the request on the data source side, return the data to Power Apps across the network, and then for Power Apps to process and display the data. Optimizing these interactions with data sources is key to great performance. The following sections highlight some of the most common mistakes.

Too many refreshes

Using the Refresh function, you can force Power Apps to update the data it has gathered from a given data source. This seems like a great function to run because you get the freshest data in your app. But, Power Apps will often handle this refresh for you. For example, when you use a Form to submit a new row to a data source displayed in a Gallery control, Power Apps will automatically refresh that connection. If you include a Refresh function when you navigate to the Gallery screen you are now refreshing the data that Power Apps already refreshed. This is redundant and slows your app down for no reason.

Note

Do not use the Refresh function until you are certain it is needed.

Too many lookups

As you start to use relational data (covered in Learning Path Use advanced data options and connectors in Power Apps -- Module 1 Work with relational data in a canvas app in Power Apps) a common mistake is not to consider the ramifications of a LookUp function inside of a Gallery. When you place a LookUp function on a Label inside of the Gallery, then that LookUp will be performed once for every row in the Gallery. That means if you have 100 rows in the Gallery the app has to perform 100 individual LookUp calls to the data source to render. Depending on the data source this could take minutes to render. A better approach is only to display the related data using a details screen or to use a Collection to cache the data from the data source, then the LookUp does not have to execute across the network.

Note

Be careful when making additional calls to remote data sources if you use controls that display multiple rows.

Storing data in the wrong data source

Different data sources are optimized for different workloads and this should be a consideration when choosing where to store data. One example is storing images or files. A common use of Power Apps is to capture images either using the Camera control or the devices built-in camera app. After the user has taken the image, it needs to be saved. One option is to store the image in the same SQL Server database as the other app data. While possible, it is important to note that SQL Server is very inefficient in storing images. Writing and reading the image file to a SQL database is slow, causing your app to run slow. A better option is to store Power Apps images in the Azure Blob Storage. Azure Blob Storage is much faster than writing the same data to SQL Server. This minor change to the underlying structure of your app can have a useful impact on user satisfaction.

Note

Choose the optimal data source for your app to get maximum performance.

Other performance considerations

While data sources might be the largest bottlenecks, there are other easily overlooked changes you can make to get optimal performance.

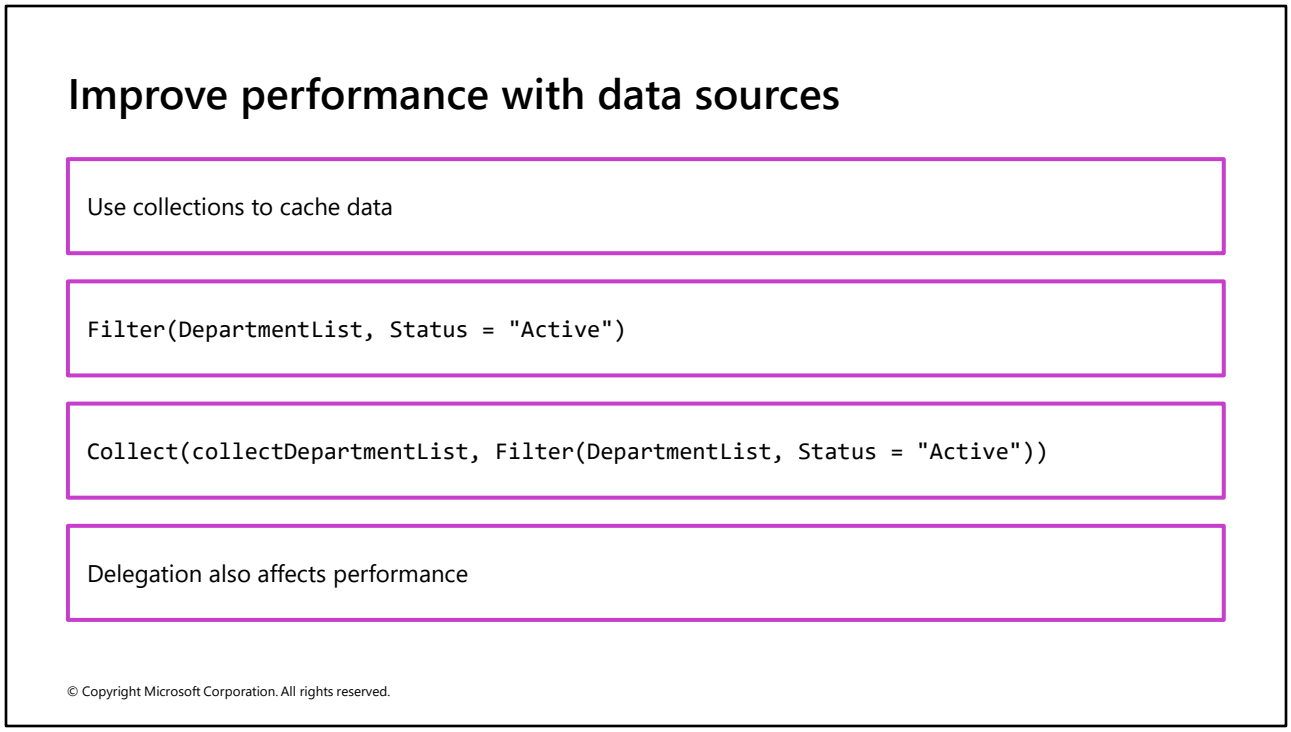
- **Asset size** – When you are designing your app it is great to include company logos and other visual assets. When you add these assets to your app, make sure the assets are optimized for the size of your app. The higher the resolution of a file, the larger the file size, and the more resources it takes for your app to store and display the image. Use an image editing tool to resize your files to the size you need for your app.
- **Republish your app** – The Power Apps team is constantly updating Power Apps to bring new features and to increase performance. The only way that your app takes advantage of these advancements is for you to open the app and publish again. Your app will stay on the version that it was published on until you do this. So periodically revisiting your app to move to the latest version will provide you with the best possible performance.
- **Build focused apps** – Power Apps supports building apps with as many screens as you can imagine, but too many screens is not a good idea. You should build your apps focused on a specific audience and process. This allows you to optimize the user experience for one audience, makes building and troubleshooting the app easier, and reduces the size of the app. If you have one app for everything, consider breaking it into smaller apps by role.

While data sources might be the largest bottlenecks, there are other easily overlooked changes you can make to get optimal performance. Some other common issues include:

- Asset size - When you are designing your app it is great to include company logos and other visual assets. When you add these assets to your app, make sure the assets are optimized for the size of your app. The higher the resolution of a file, the larger the file size, and the more resources it takes for your app to store and display the image. Use an image editing tool to resize your files to the size you need for your app.

- Republish your app - The Power Apps team is constantly updating Power Apps to bring new features and to increase performance. The only way that your app takes advantage of these advancements is for you to open the app and publish again. Your app will stay on the version that it was published on until you do this. So periodically revisiting your app to move to the latest version will provide you with the best possible performance.

- Build focused apps - Power Apps supports building apps with as many screens as you can imagine, but too many screens is not a good idea. You should build your apps focused on a specific audience and process. This allows you to optimize the user experience for one audience, makes building and troubleshooting the app easier, and reduces the size of the app. If you have one app for everything, consider breaking it into smaller apps by role.



Improve performance with data sources

•12 minutes

In the previous unit, you learned that data sources are often the main reason for slow performance in your app. In this unit, you will learn about some of the common techniques you can apply to mitigate those performance issues.

Use collections to cache data

Often in your app you will find yourself query the same data repeatedly, like in the case of pulling the lists of departments for your drop-down menus. In those instances, you can query for the data once and then reuse that data throughout the app. This reduces the repetitive calls to the data source across the network. The following is an example of this process.

In your app, you have several screens where you provide a drop-down menu for selecting the department. The list of departments is kept in Microsoft Dataverse in a table named **DepartmentList**. On each instance of the menu, you use the following formula.

Copy

Filter(DepartmentList, Status = "Active") This works and performs okay.

When you finish the app, you realize that you are querying the **DepartmentList** table multiple times even though it is static information. You could modify your app to create a collection with the **OnStart** property of the app that stores the information using the following formula.

Copy

Collect(collectDepartmentList, Filter(DepartmentList, Status = "Active")) Now that you have stored that information you could change the **Items** property of the drop-down controls to be **collectDepartmentList** instead of Filter(DepartmentList, Status = "Active"). These small changes add up to increase performance in your app. Also, as you become more familiar with the technique, you can build your app this way from the onset which reduces the number of formulas you have to write and maintain.

Watch out for delegation

One thing to keep in mind is that the Collect function is not delegable. By default, that means only the first 500 items will be returned from your data source and stored in the collection. Be sure to plan for this limitation as you implement the use of collections in your app.

Delegation also affects performance

When you learned about delegation, you focused on returning the right amount of rows for your data source. It is also important to remember that delegation, especially for mobile apps, can affect performance.

When a formula delegates to the data source, all of the processing is handled by the data source, and only the matching rows are returned across the network to the app to be displayed. If a function is not delegable, then it is very common to change the delegation limit to 2,000 rows. This means that the first 2,000 rows will be downloaded across the network and then processed locally. In scenarios where you are on a slow cellular connection or a low-end mobile device this processing can take a considerable amount of time, causing a poor experience for the user.

Try to use only delegable functions as much as possible. If your function is not delegable, then plan for the impact on the end user.

Use the Concurrent function to load multiple data sources

```
Collect(collectDepartmentList,
Filter(DepartmentList, Status = "Active"));

Collect(collectCompanyList, CompanyList);

Collect(collectRegions, RegionList);
```

```
Concurrent(

Collect(collectDepartmentList,
Filter(DepartmentList, Status = "Active")),

Collect(collectCompanyList, CompanyList),

Collect(collectRegions, RegionList)

)
```


Previously you learned to use collections to cache data in your app. As you implement that functionality in your app it is not uncommon to have several collections load when the app launches. Your OnStart property might look like the following.

In that example, you are creating 3 collections, but the collections are processed one at a time. So if each one takes 3 seconds to process then your user has to wait 9 seconds for the app to start.

The Concurrent function allows you to process all of those calls at the same time. You can change your code to the following.

Now all three formulas run at the same time. Reducing your load time to 3 seconds. Concurrent is a great way to avoid long delays from asynchronous calls.

OnStart versus OnVisible

OnStart and OnVisible are part of your toolkit for building great apps, but from a performance point of view, they can have a major impact.

- **OnStart** – This is an app-level property. Formulas in this property are run once, when the app starts, and never again. All of the formulas must process before the app opens. This is often used to initialize data that you will need throughout the app.
- **OnVisible** – This is a per-screen property. Formulas in this property are run every time a user navigates to the screen. The screen will render before the formula is finished processing.

OnStart and OnVisible are part of your toolkit for building great apps, but from a performance point of view, they can have a major impact.

- OnStart - This is an app-level property. Formulas in this property are run once, when the app starts, and never again. All of the formulas must process before the app opens. This is often used to initialize data that you will need throughout the app.
- OnVisible - This is a per-screen property. Formulas in this property are run every time a user navigates to the screen. The screen will render before the formula is finished processing.

From a performance perspective, the key consideration is when the code runs. Once, when the app starts, or every time a screen is shown. The collection for departments from earlier in this module is a great example. That code loads in the OnStart property of the app. This means it was loaded once and then always available. If you moved that code from OnStart to OnVisible, you would lose the advantage of the collection. If the formula was in the OnVisible property, each time the user navigated to the screen the data source would be queried. In that case, it would perform just as well to leave the code in the drop down.

This doesn't mean you should not use OnVisible. OnVisible is great for formulas that pertain to the current screen that you need to run. Additionally, OnVisible is non-blocking, so your users don't have to wait on the formula to complete to view the screen, which reduces the amount of that user needs to view a blank screen.

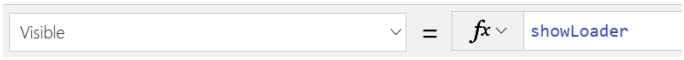
In most apps, you use a mixture of OnStart and OnVisible to get the optimal experience.

Optimize app load time

Maker portal, includes the **Time to first screen** and **Time to first screen without connection setup** analytics

Review app settings

Use a loading spinner image



Delegation also affects performance

First impressions are important. The first impression that app users get is the length of time that it takes for the app to display the first screen or to provide other visual feedback, such as progress indicators. During startup, your app performs several steps to prepare for the presentation of the first screen: Authenticate, get metadata, **OnStart** processing, and render screens.

Evaluate your app load time

When you're evaluating app load time, a good place to start is establishing a baseline for how long it takes for your app to load. You can accomplish this task by measuring the startup time from launching the application to the time when you believe that you have a usable app. Use a stopwatch to measure the time from launching the app to when you believe a person can start using it. Tools that you can use to measure and drill into the performance details during the load time are discussed later in this module. We recommend that you conduct a new evaluation with each app update so that you can compare the new version with your prior baseline and then identify significant increases in load time.

If you view analytics for your app from the maker portal, it will include the **Time to first screen** and **Time to first screen without connection setup** analytics.

Review your app settings

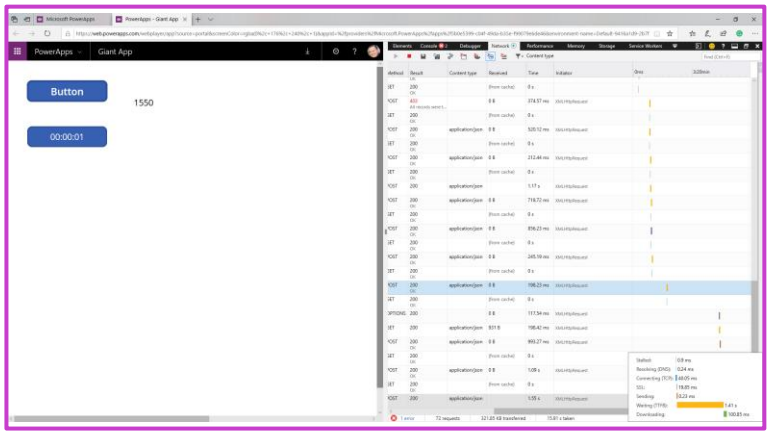
App settings can have a significant impact on the performance of your app; therefore, make sure that you review the app settings and their enabled status with each app update. Older apps might not have the latest app setting options enabled automatically to ensure that the new option doesn't break existing apps. You might have also enabled a setting to try resolving a problem but then forgot to turn off the setting when you were done. A good example of that scenario is the **Debug published app** option. Enabling this option allows more debug information to be displayed when you use the app monitor to troubleshoot the published app. When this option is enabled, it can be detrimental to the performance and should not be left on for production use.

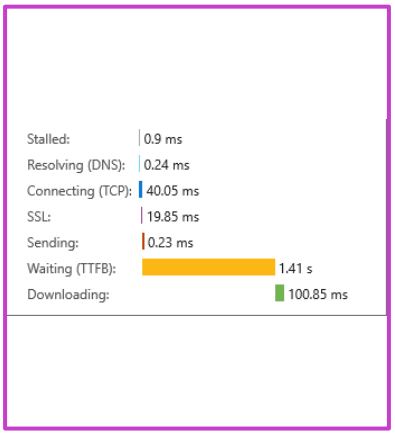
Another setting to consider is **Data row limit**. This setting determines the most rows that will be retrieved from a server-based connection when delegation is not supported. By default, this value is 500 and can be any value between 1 and 2,000. To work around delegation issues in apps, it's common for people to increase this value. This increase might cause unexpected problems when a development environment holds much smaller sets of data than production. For example, with the **Data row limit** set to 2,000, the following expression in **Data row limit** might only preload a few rows in a development environment.

Using a loading image or a progress indicator during long operations in your app can improve user perception.
Then, in **App.OnStart**, you can turn on the loading image before loading the data, and then you can turn off the image after the data is loaded.
Set(showLoader, true); ClearCollect(colDesks, Desks); ClearCollect(colDeskFeatures, 'Desk Features'); Set(showLoader, false);

Looking at the network activity of your app

Now that you have learned about testing from within the app you need to look at actual network calls and performance.





Now that you have learned about testing from within the app you need to look at actual network calls and performance. To do this you can use your browsers built-in developer tools, Ctrl+Shift+I in Chrome or F12 in Edge/IE, or a third-party tool like Fiddler. This will allow you to view the individual network calls made by your app and view details, such as time that each call takes. From a performance point of view, this can be valuable.

One example of a way you can use this is to determine if performance slowness, as measured by the Timer control from the previous example, is within your app, the network, or the data source. By using the Developer Tools in Microsoft Edge, you can see the breakdown of the query that took 1.55 seconds in Power Apps.

By hovering over the query in the bottom right of the screen, you can see the time breakdown as follows.

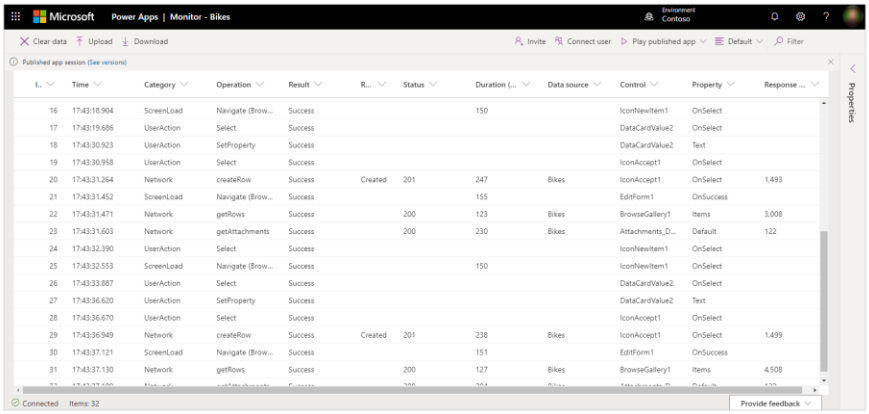
In this instance, most of the time was spent waiting on the data source to filter the data and respond. This tells you that you cannot make the call faster by changing the app. Instead, you would need to focus on refining the query or speeding up the data source.

This level of information can be overwhelming at first but don't try to learn the whole interface of the developer tools, instead, concentrate on just the pieces you need like the network timelines.

Additionally, you can use the developer tools to get more details about error messages returned in the app or a stronger indication where connectivity issues are occurring.

Monitor

Monitor is a tool that you can launch from Microsoft Power Apps Studio to help you troubleshoot problems and improve the quality of your apps

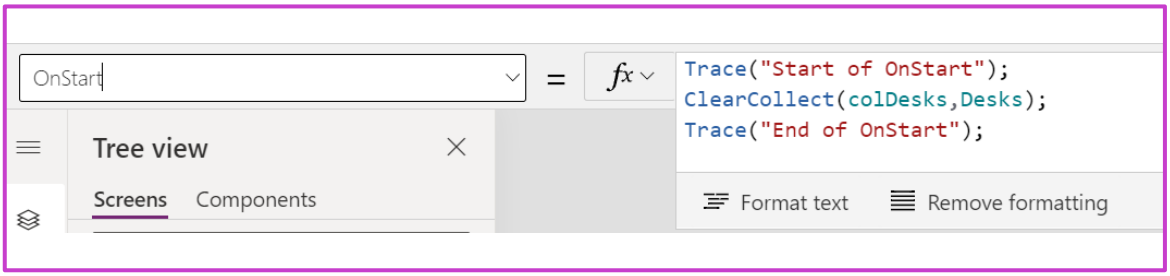


Key elements that you can identify by using Monitor include:

- Errors in using connectors
- Extensive data being sent/received
- Slow response from connectors
- Duplicated data actions

Trace

Add custom trace messages for Monitor



In addition to the events being automatically captured, you can also log custom messages by using the Trace() function. The custom messages can be helpful in marking the start or end of **OnStart** or **OnVisible** logic. The following example shows the process of adding a Trace() function before and after data is preloaded from Microsoft Dataverse.

Learning Path 1 practice labs

Power Apps

Lab 2: Advanced canvas app techniques

As we continue to build our solution, we will now customize the canvas app to use variables and the Patch formula.

Summary

- Canvas apps support both imperative and declarative logic
- Use declarative techniques where possible
- Global variables are for storing information to use throughout your app
- Context variables are only available on the same screen
- Collections are a type of variable that stores table data
- The Patch function is the multi-purpose function for creating and editing records directly in your data source when forms don't work for your scenario
- The dot notation helps you leverage one-to-many relationships
- Understand when delegation is and isn't available
- Interactions between the data source and the app are the most common cause of poor performance
- Optimize app load time
- Use Monitor to debug and troubleshoot canvas apps

Modules

1. Use imperative development techniques for canvas apps
2. Perform custom updates in a canvas app
3. Use Dataverse choice columns with formulas
4. Work with relational data in a canvas app
5. Work with data source limits (delegation limits) in a canvas app
6. Performance in canvas apps

Microsoft